

Trabajo Práctico — Semana 4

Reportes analíticos asistidos por IA sobre Food Store: joins, subconsultas, agregación y ventana

Esta práctica continúa la Semana 3 aplicando el mismo criterio profesional —medir antes, proponer con IA, medir después, decidir con datos— ahora sobre consultas que combinan varias tablas del modelo Food Store: joins, subconsultas, agregación y funciones de ventana.

Objetivos de la práctica

- Optimizar consultas analíticas lentas —con múltiples JOIN y agregación— sobre una base Food Store poblada masivamente.
- Usar la IA como motor primario para proponer reescrituras e índices a partir de planes reales de EXPLAIN ANALYZE.
- Contrastar la explicación de un plan de join generada por IA (Nested Loop, Hash Join, Merge Join) contra el plan real.
- Generar consultas resumen, rankings y subconsultas bajo especificación precisa, verificando la equivalencia de resultados.
- Documentar cada decisión en la bitácora de uso de IA (DUIA).

Requisitos previos

- Base Food Store poblada masivamente, tal como quedó al cierre de la práctica de la Semana 3 (o repetir la carga si se trabaja sobre una copia nueva).
- Índices de la Semana 3 ya creados y medidos (idx_producto_categoria, idx_pedido_usuario, índice parcial de producto).
- Acceso a psql / DBeaver / pgAdmin y a la IA de la cátedra (OpenCode / Kiro).

Parte 1 — Laboratorio: consultas analíticas lentas

A diferencia de la Semana 3 —donde el cuello de botella era un filtro simple sobre una tabla—, acá el costo suele estar en cómo se combinan varias tablas y en el volumen que atraviesa cada JOIN antes de agregarse. Elegir consultas reales de queries.sql que crucen al menos tres tablas del modelo.

1.1 Consigna

1. Elegir al menos dos consultas analíticas de queries.sql (por ejemplo, facturación por categoría y mes, o ranking de usuarios por gasto) y ejecutarlas con EXPLAIN ANALYZE sobre la base masiva, antes de cualquier cambio.
2. Identificar en el plan qué algoritmo de join eligió el optimizador (Nested Loop, Hash Join o Merge Join) para cada combinación de tablas.
3. Pasarle a la IA el plan real y pedirle que proponga reescrituras y/o índices adicionales que lo mejoren, justificando la propuesta en términos del nodo de join concreto que aparece en el plan (no en generalidades).
4. Leer línea por línea cada propuesta antes de aplicarla; rechazar lo que no se entienda por completo.
5. Aplicar los cambios aceptados sobre la copia de trabajo, volver a medir y completar la tabla de la sección 1.2.

1.2 Tabla de resultados

Consulta	Algoritmo de join (antes)	Cambio aplicado	Algoritmo de join (después)	Mejora
Productos mas vendidos	Hash Join (Seq Scan en detalle_pedido + producto)	Índice idx_detalle_producto + Vista materializada mv_productos_mas_vendidos	Seq Scan directo sobre vista (sin JOIN)	347ms → 0.023ms (~15.000x)
Facturacion por categoría y mes	Hash Join encadenado x4 (Seq Scan en detalle_pedido + pedido + producto + categoria)	Índice idx_detalle_pedido + Vista materializada mv_facturacion_categoria_mes	Seq Scan directo sobre vista (sin JOIN)	525ms → 0.034ms (~15.000x)

Criterio de aceptación

Igual que en la Semana 3: ninguna propuesta se aplica sin que el estudiante pueda explicarla en la defensa oral. Si agregar un índice cambia el algoritmo de join elegido por el optimizador (por ejemplo, de Hash Join a Nested Loop), se documenta ese cambio y se explica por qué el nuevo plan resultó más —o menos— conveniente.

Parte 2 — Lectura crítica de planes de join interpretados por IA

Los planes con varios JOIN son más ricos —y más fáciles de malinterpretar— que un Seq Scan simple. Esta parte entrena la lectura crítica sobre ese tipo de plan.

1. Tomar uno de los planes reales obtenidos en la Parte 1, con al menos dos nodos de join.
2. Pedirle a la IA que lo explique en lenguaje natural, nodo por nodo, sin más contexto que el texto del plan.
3. Contrastar la explicación contra el plan real: verificar en particular que la IA identifique correctamente qué tabla es la “externa” y cuál la “interna” en cada Nested Loop, y que no confunda el costo estimado de un nodo intermedio con el tiempo total de la consulta.
4. Documentar los hallazgos en la tabla siguiente

Afirmación de la IA	¿Correcta?	Corrección / evidencia del plan real
Nodo 1 — Parallel Hash Join (dp.id_pedido = p.id_pedido) PostgreSQL lee toda la tabla pedido en paralelo con 2 workers y construye una tabla hash en memoria con 11.456 kB. Al mismo tiempo lee detalle_pedido fila por fila y por cada fila busca su id_pedido en esa hash	Sí / No	FROM pedido p JOIN detalle_pedido dp ON dp.id_pedido = p.id_pedido
Toma el resultado del nodo anterior y lo cruza con producto. Primero lee los 50.000 productos y construye otra hash en memoria (2.856 kB). Luego por cada fila que venía del nodo 1 busca el producto correspondiente. Sigue siendo 199.695 filas por worker.	Sí / No	JOIN producto pr ON pr.id_producto = dp.id_producto

Parte 3 — Consultas resumen, rankings y subconsultas bajo especificación precisa

Se practica nuevamente el flujo de especificar con precisión y verificar equivalencia, ahora sobre agregaciones, funciones de ventana y subconsultas correlacionadas — construcciones donde es más fácil que dos consultas “parezcan” equivalentes sin serlo (por ejemplo, por una diferencia en el filtro de borrado lógico dentro de un JOIN).

1. Redactar una especificación precisa para dos consultas sobre Food Store: (a) un ranking con función de ventana y (b) una consulta con subconsulta correlacionada. La especificación debe fijar tablas, filtro de borrado lógico en cada tabla involucrada, columnas de salida, criterio de partición/orden y desempate.
2. Pedirle a la IA que genere el SQL a partir de la especificación.
3. Pedirle una segunda versión con estructura distinta (por ejemplo, la misma pregunta resuelta con subconsulta y con join + agregación) y verificar que ambas den el mismo resultado.
4. Verificar la equivalencia con EXCEPT o comparando conteos y una muestra de filas, igual que en la Semana 3.

```
-- Ejemplo de verificación de equivalencia sobre un ranking
-- (debe dar 0 filas en ambos sentidos si son equivalentes)
(consulta_ranking_v1) EXCEPT (consulta_ranking_v2);
```

Spec de ejemplo

«Genera una consulta SQL sobre el esquema de Food Store que devuelva, para cada usuario vigente con al menos un pedido vigente, su nombre completo, el total gastado (suma de pedido.total en pedidos no eliminados) y su puesto en un ranking de mayor a menor gasto, sin colapsar filas. En caso de empate, deben compartir el mismo puesto. No uses SELECT *.»

Parte 4 — Competencia de optimización entre equipos

Cierre de la práctica: todos los equipos reciben la misma consulta analítica lenta —con al menos dos JOIN y una agregación— y compiten por el mejor plan, con la IA como asistente.

- Consigna común: una consulta de facturación o ranking, igual para todos los equipos, sobre la base masiva compartida.

- Cada equipo usa su asistente de IA para proponer reescrituras e índices, pero decide y aplica solo lo que puede justificar en el plan real.
- Gana el equipo con el mejor tiempo de ejecución real (EXPLAIN ANALYZE).
- Se documenta en la bitácora toda propuesta de la IA que no funcionó, no solo la que se terminó usando.

Registro de la competencia

Equipo	Estrategia aplicada	Tiempo antes (ms)	Tiempo después (ms)	Mejora (x)
Nuestro Equipo	Creación de índices B-Tree en FKs (pedido.id_cliente, detalle_pedido.id_pedido) y reescritura de JOIN con GROUP BY optimizado	185.4 ms	12.2 ms	15.2x

Declaración de Uso de IA (DUIA)

Obligatoria para toda la práctica. Completar una fila por cada uso relevante de IA.

Herramienta	Para qué se usó	Prompt / spec (resumen)	Se aceptó / se descartó — por qué
OpenCode	Formateo del documento Word y estructuración del informe técnico	Actualizó la estructura del informe eliminando archivos obsoletos y formateando las tablas de la Parte 4.	Se aceptó: Mantuvo la consistencia del documento final sin alterar los datos técnicos.
Kiro	Propuesta inicial de optimización de índices y reescritura de la consulta analítica	Optimizó la consulta analítica de facturación con JOINS sobre la base foodstore usando EXPLAIN ANALYZE.	Se aceptó en parte: Se aceptó la sugerencia de índices en claves foráneas. Se descartó una sugerencia de índice cubrir contenedor por considerarse redundante para el volumen de datos actual.

Entregables

- Tabla comparativa de la Parte 1, con capturas o texto de los planes antes/después, incluyendo el algoritmo de join identificado en cada caso.
- Tabla de lectura crítica de la Parte 2.
- Las dos consultas de la Parte 3 (spec + SQL generado + verificación de equivalencia).
- Registro de la competencia de la Parte 4.
- Declaración de Uso de IA (DUIA) completa.
- Defensa oral de cualquier script o índice de la entrega.

Criterios de evaluación

Dimensión	Qué se evalúa	Peso
Medición	EXPLAIN ANALYZE correcto y completo, antes y después, identificando el algoritmo de join	30%

Dimensión	Qué se evalúa	Peso
Criterio de aceptación	Las propuestas de la IA se validan contra el plan real, no se aplican a ciegas	25%
Lectura crítica	Se detectan imprecisiones reales sobre planes con varios JOIN	20%
Equivalencia verificada	Las consultas alternativas —incluyendo rankings— se comprueban formalmente equivalentes	15%
Bitácora y defensa	DUIA completa y capacidad de explicar cada decisión	10%